

category: dotnet-general

tags: [async, race-condition, ui-counter, dashboard, timer, avalonia]

severity: medium

created: 2026-07-30

updated: 2026-07-30

project-origin: App-Access-V2 (iAccessDesktopv2.Avalonia)

KPI đếm theo trạng thái tạm thời set-rồi-reset đồng bộ → luôn đọc 0

Tình huống gặp phải

Dashboard Avalonia có KPI card "Lệnh chờ xử lý" (pending commands), tính bằng cách đếm số dòng có `Status == "Đang thực hiện"` trong 1 `ObservableCollection` hiển thị log xử lý lệnh (`RegisterStatuses`), refresh theo `DispatcherTimer` riêng trên UI.

Triệu chứng / Lỗi

KPI luôn hiển thị 0, kể cả khi thực tế đang có lệnh chờ xử lý từ server — không có exception, không có warning, build sạch.

Nguyên nhân gốc rễ (Root Cause)

Nơi sinh ra "trạng thái tạm thời" và nơi đọc nó nằm trong 2 timer khác nhau, nhưng trạng thái đó chỉ tồn tại trong đúng 1 method chạy đồng bộ-tuần tự (await nối tiếp, không yield về caller giữa chừng):

```
var row = new RegisterStatusRow(..., "Đang thực hiện");
await Dispatcher.UIThread.InvokeAsync(() => RegisterStatuses.Add(row));
var result = await controller.RegisterUser(data, ...); // xử lý xong ngay trong cùng
method
...
await Dispatcher.UIThread.InvokeAsync(() => row.Status = "Xong"); // set lại NGAY sau
đó
```

Toàn bộ transition "Đang thực hiện" → "Xong" diễn ra trong 1 lần gọi hàm của timer A (timer xử lý lệnh, VD 1-2s/tick). Timer B (timer refresh Dashboard KPI, đọc `RegisterStatuses`) chạy độc lập và gần như không bao giờ tick đúng vào khoảnh khắc cực ngắn giữa 2 dòng trên → count luôn ra 0.

Bài học tổng quát: Bất kỳ KPI/counter nào tính bằng cách đếm 1 "trạng thái hiển thị tạm thời" (status string set rồi reset trong cùng 1 hàm đồng bộ) sẽ **không phản ánh đúng** con số thực — vì trạng thái đó không tồn tại đủ lâu để một timer/observer độc lập khác kịp đọc.

Giải pháp

Đếm trực tiếp từ **nguồn dữ liệu bền** phản ánh đúng "còn việc chưa xử lý" — ở đây là

`ConcurrentQueue<T>` (`AccessControllerClient.RequestQueues`) chứa các lệnh CHƯA được dequeue — thay vì đếm theo dòng log hiển thị:

```
// SAI - đếm theo Status transient, luôn ra 0
PendingKpi.Value = vm.RegisterStatuses.Count(r => r.Status == "Đang thực hiện").ToString();

// ĐÚNG - đếm theo hàng đợi thật (item chưa dequeue), phản ánh đúng số lệnh đang chờ
PendingKpi.Value = vm.Controllers.Sum(c => c.RequestQueues.Count).ToString();
```

1. Xác định "trạng thái transient" nào đang bị đếm nhằm làm nguồn số liệu.
2. Tìm nguồn dữ liệu bền (queue, DB row chưa xử lý, flag không tự reset) phản ánh đúng ý nghĩa business của con số cần hiển thị.
3. Đổi KPI sang đếm từ nguồn bền đó — không phụ thuộc timing giữa 2 timer độc lập.

Áp dụng lại (How to reuse)

- Khi thấy 1 KPI/counter dashboard luôn = 0 (hoặc luôn = giá trị cố định) dù chắc chắn có dữ liệu thực tế → nghi ngay race condition giữa timer sinh dữ liệu và timer đọc dữ liệu, không phải bug ở công thức đếm.
- Trước khi tin 1 status string ("Đang thực hiện"/"In Progress"/...) là nguồn đếm đáng tin, kiểm tra: trạng thái đó có bị set-rời-reset trong CÙNG 1 method đồng bộ không? Nếu có → không dùng để đếm, tìm nguồn bền hơn (queue count, DB flag, timestamp range).
- Grep tên field Status + tìm mọi chỗ gán giá trị của nó để xác nhận vòng đời thực tế trước khi dùng làm điều kiện đếm.

Chú ý / Cạm bẫy (Gotchas)

- ⚠ Bug loại này KHÔNG lộ qua build/test đơn giản vì logic "trông đúng" — chỉ lộ khi quan sát UI runtime qua nhiều tick.
- ⚠ Khi sửa, kiểm tra thêm có đang gán sai "**Xong**" cho CẢ trường hợp Fail hay không (root cause khác, dễ bị bỏ sót cùng lúc vì nằm cạnh nhau trong code) — không tự sửa nếu không được yêu cầu, nhưng nên ghi chú lại cho task sau.

Tham chiếu

- Project liên quan: App-Access-V2 (KpiRowView.axaml.cs, MainViewModel.cs — RunCMDFromServerAsync)